CSD457 INTERNET OF THINGS · GROUP G-04 · OCTOBER 2026

Progress report: Poison-Resilient On-Device Intrusion Detection for Drifting IoT

Rishit Kamboj and Farhan Naik · Changes since the proposal deck, progress, and what we will build and test next.

The project in one paragraph

An IoT device that learns what normal traffic looks like can be taught, slowly, that attacks are normal. We are building a detector that **keeps learning on an ESP32** and cannot be steered that way. Our first method rationed the part of each update that moves toward a known attack. New 2026 papers prove that a detector watching one stream cannot reliably tell honest change from slow poisoning, so the next step gives each device a **second source of evidence: its neighbours**. Devices on the same network see the same honest drift; an attacker usually reaches only one.

Where we stand

Area	Status	One line
Idea and literature	Done	3 research rounds; 36 references compiled, 26 from 2025 to 2026 (last page)
Method v1: Threat-Orthogonal Adaptation (TOA)	Done	Works on synthetic and real N-BaIoT traffic
Method v2: signature-free version	Done	Beats a normal and a smart attacker on real traffic; fails if the attack hides inside strong honest drift
Freeze attack on published defence designs	Done	Tested on three published gate mechanisms (simplified re-implementations): one is lockable (page 2)
Method v3: peer-witnessed adaptation (FOA)	Designed	Mechanism and tests written; not yet implemented
Reproducible real data	Done	get_nbaiot.py downloads just the files we need
ESP32 hardware	Not yet	No measurements yet; parts list and wiring on pages 4 to 5

What changed since the proposal deck (24 September)

- **Real traffic.** Real N-BaIoT doorbell and Mirai packets added. Naive learning is poisoned at 10 to 20% attacker share.
- **Signature-free method** added, so TOA no longer needs a known attack signature. Its weakness (an attack hidden in strong honest drift) is, per three 2026 papers, a proven limit of any single-stream detector; the way out is outside evidence.
- **Novelty narrowed honestly:** ANTIDOTE (2009) already defends a server-side detector, and every direction we checked was only partly new, so we claim a specific combination (page 3).
- **Freeze attack now tested on published defence designs,** not just our own baselines.

Corrections and decisions

The 10% figure holds only for the scan attack (20% for ack and syn). The 52% traffic-drop figure came from an outside analysis of the CESNET data, not from the CESNET paper. **We need:** how many ESP32 boards and INA219 modules we have, and the real mid-review and final dates.

RESULTS

What we have shown so far

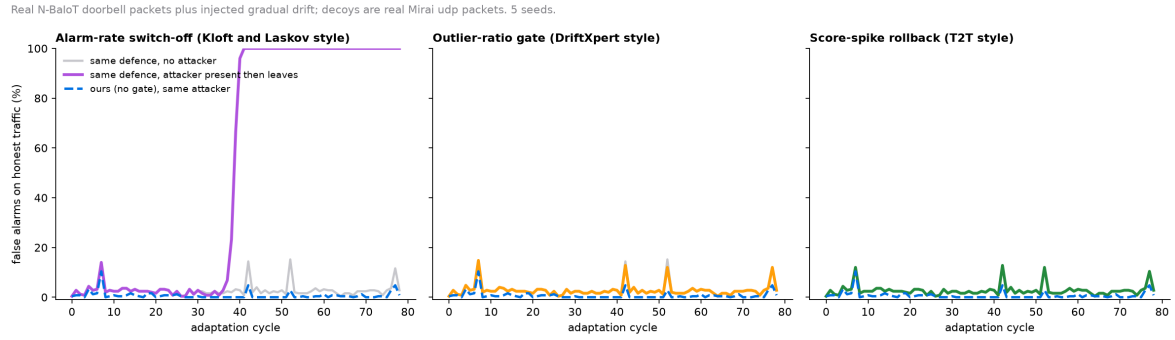
Test	Result
Synthetic, attacker at 40% of traffic	Naive: attack hidden after 24 cycles. Freeze-style defences: 85 to 87% false alarms. TOA: 100% detection, 0.3% false alarms
Real doorbell, boundary attacker, 10 to 50% share	Naive fails at 10% (scan) or 20% (ack, syn). TOA and signature-free: 100% detection, 0.8% to 1.5% false alarms
Real traffic, smart attacker hiding poison in the core	Signature-free: 100% detection, 0.2% to 0.7% false alarms
Strong honest drift and attack together (synthetic)	Signature-free still poisoned in 3 of 4 settings (2% to 46% detection): the open problem
New device joins (real thermostat traffic)	All detectors about 34% false alarms: one shared detector cannot absorb a new device

New: the freeze attack on published defence mechanisms

Many published defences protect themselves by **withholding updates when something looks suspicious**. We re-implemented the gating rule of three of them around our detector (simplified versions of the mechanism, not the full systems; rule details to be checked against the PDFs). The attacker sends only **loud decoy traffic that never poisons anything**. It holds a gate closed while honest drift begins, then leaves. If the detector cannot follow the drift, its own false alarms keep the gate shut.

Defence mechanism	No attacker, false alarms at the end (synthetic / real)	After the attacker left (synthetic)	After the attacker left (real packets)	Verdict
Alarm-rate switch-off (Kloft and Laskov, JMLR 2012, style)	1.3% / 2.7%	52%	100%	**Locked out**
Outlier-ratio gate (DriftXpert, 2026, style)	1.3% / 2.7%	1.2%	2.7%	Recovers
Score-spike rollback (T2T, 2026, style)	1.3% / 2.7%	1.2%	2.7%	Recovers
Ours (no gate)	1.1% / 0.8%	1.1%	0.8%	Unaffected

How much attacking it takes. Against the alarm-rate gate, the lock-in is a threshold effect: too short a push and the gate reopens and the detector catches up. Holding the gate for about 40 cycles (synthetic) or 15 cycles (real, faster drift) locked it in **10 of 10 runs**, with decoys as low as 5% of traffic (about 200 to 600 packets). Gates whose baseline moves with the traffic (outlier ratio) or that only roll back two cycles (T2T) were not vulnerable as modelled. A fourth, confident-sample stop rule (RTTAD style), broke even with no attacker, so our simplified version says nothing about the attack and is excluded.



Real N-BaIoT doorbell packets with an injected gradual drift; decoys are real Mirai udp packets; 5 seeds. Purple: attacker present then leaves. Grey: same gate, no attacker. Blue dashed: ours.

DIRECTION

What our research is now about

In short: Fleet of ESP32 detectors that keep learning honest change but cannot be steered, using neighbouring devices as witnesses, and measure what it costs on real hardware.

Research questions

- **RQ1.** Can peer witnesses let a device keep learning honest drift while resisting poisoning, where a single-stream detector provably cannot?
- **RQ2.** Which published defences fall to a freeze attack, and does never-stopping adaptation avoid it? (Answered in simulation; to repeat on the ESP32.)
- **RQ3.** What does it cost on an ESP32 in memory, latency, radio bytes and energy?

Method: Fleet-Orthogonal Adaptation (FOA)

Each device keeps its own centre-and-spread detector. Every cycle it proposes an update from the dense core of its own traffic and shares it (10 numbers, signed, about 72 bytes) with peers over ESP-NOW. The peers' geometric median is the **common part** (network-wide drift); the rest is the **residual**.

- Common part: applied in full, so honest drift is learned and the detector never freezes.
- Residual: goes through TOA's slow budget, so a campaign only one device sees is rationed.
- Peers differ in vendor or firmware, so one exploit cannot make several devices agree.
- A new device starts from its peers' median and learns under the residual budget, bounding join-time poisoning.

Closest published work	How we differ
CIoTA (NDSS DISS 2018, JPDC 2020): same-firmware devices must agree; disagreement is discarded	Diverse peers, so a one-exploit botnet cannot fake agreement; the disagreeing part is admitted slowly, not discarded
WARDEN/CDC and two other 2026 theory papers: single-stream limits	A second stream escapes them; we must prove that as our own theorem
Gate-based defences (Kloft and Laskov, DriftXpert, T2T, OWAD)	FOA has no stop gate; the freeze-attack study above shows why that matters

Every candidate direction was only partly new in our prior-art checks, so the claim is a specific combination, not a first.

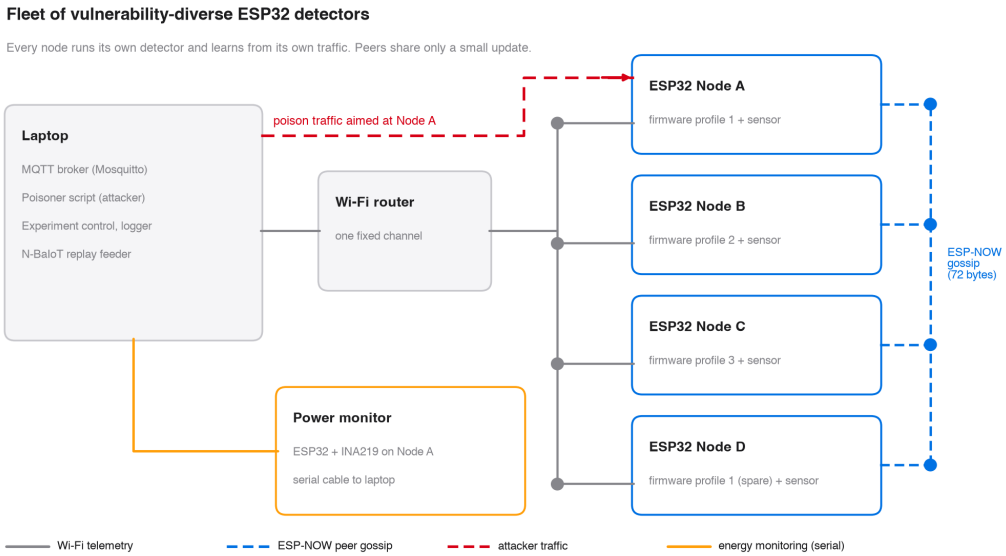
What we will test

#	Test	Against	We win if	Status
1	Attack hidden inside strong honest drift	Signature-free single stream; WARDEN-style frozen reserve	FOA keeps detection where single-stream fails	**Week 1**
2	Monoculture botnet reaches several same-firmware peers	CIoTA-style same-firmware consensus	Diverse peers hold; same-firmware consensus is fooled	**Week 1**
3	Freeze attack on published gates	Kloft and Laskov, DriftXpert, T2T styles	Gated defences lock out; FOA does not	**Done in sim**
4	New device joins; adaptive attackers (core-hiding, pulsing, omission)	One shared detector; naive	False alarms near nominal; no attacker beats the budget	**Week 2**
5	ESP32 cost: RAM, latency, radio bytes, energy	Naive detector on the same board	Overhead small enough to justify	**Week 3**

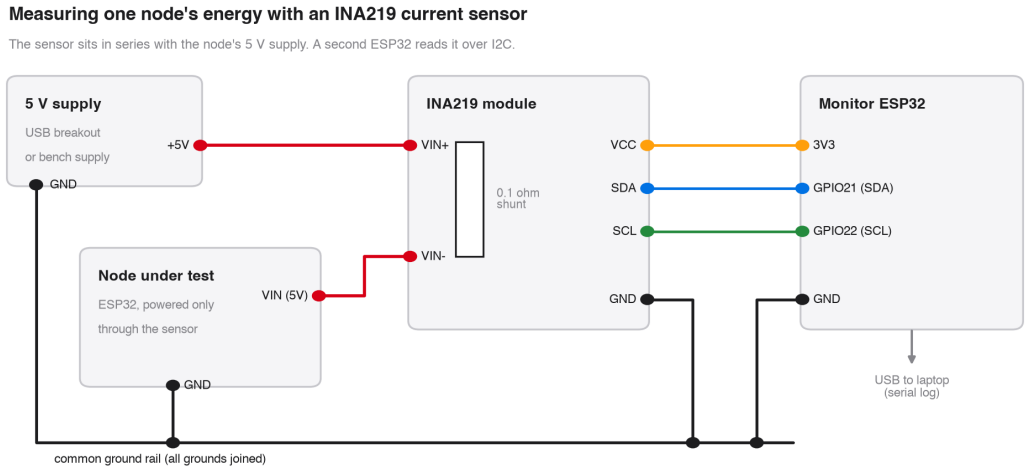
Data: N-BaIoT doorbell, thermostat and cameras as peers (devices were recorded separately, so shared drift is injected and labelled synthetic); the real ESP32 fleet with a router change supplies genuine shared drift. Headline figure: under a monoculture botnet, same-firmware consensus is compromised while diverse peers hold; plus false alarms after the attacker leaves.

HARDWARE

IoT set-up and wiring



Four ESP32 nodes with different firmware share a Wi-Fi network and gossip small signed updates over ESP-NOW. The laptop runs the MQTT broker, the poisoner and the logger. A separate ESP32 reads the INA219 on Node A.



Build notes (check before wiring):
1. Add 4.7 kohm pull-ups from SDA and SCL to 3V3 if the INA219 module has none.
2. Flash the node first, then power it only through this path. Do not also plug in its USB at the same time.
3. INA219 samples about every 0.5 ms, so measure energy over many cycles and average. Do not trust single short bursts.

Proposed wiring, not yet built. Check every pin against your module's datasheet before powering it. Default I2C pins on the ESP32 DevKitC are GPIO21 (SDA) and GPIO22 (SCL).

PARTS AND PLAN

What we need, and the next four weeks

Item	Qty	Why
ESP32 development boards (for example ESP32-DevKitC)	4, plus 1 for monitoring	Fleet nodes A to D and the power monitor
INA219 current-sensor module, 5 V supply or USB breakout	1 to 2	Measures a node's power and energy
BME280 or DHT22 sensor (optional)	1 per node	Gives each node real telemetry to send
Breadboard, jumpers, 4.7 kohm resistors, USB cables and hub	1 set	Wiring, I2C pull-ups, flashing, serial logs
Wi-Fi router on a fixed channel; laptop	1 each	ESP-NOW and Wi-Fi share one channel; laptop runs broker, poisoner, logger

Software: ESP-IDF or the Arduino core (ESP-NOW and mbedTLS HMAC are standard), Python and NumPy, Mosquitto, and our experiment scripts. Two run modes: **replay** (N-BaIoT features fed over serial, for reliable cost numbers) and **live** (the board's own telemetry). Measured on the board: peak RAM and flash, latency per update and per signed message, ESP-NOW bytes per learning event, and energy per update from the INA219. Our estimate is about 1,000 multiply-adds and under 10 KB per packet on a 240 MHz, 520 KB part; that is an estimate until we measure it.

Four-week plan

Week	Work	Output
1	Implement FOA in simulation: geometric median, signed gossip, two cumulative budgets, join rule. Add the CIoTA-style baseline. Run tests 1 and 2. Read the three unread papers. Order or check parts	FOA results in simulation
2	Attacker matrix and test 4. Build the headline figure and the mid-review pack. Port the detector to the ESP32 in replay mode. Wire the INA219	First real results; board running
3	Bring up the ESP32 fleet with FOA over ESP-NOW. Router change as real shared drift. Measure RAM, latency, bytes and energy (test 5)	Hardware demo and cost numbers
4	Proofs as a stated assumption plus the matching negative case; ablations (peer count, budget, diverse vs random peers); IEEE report, code package with README, demo	Final deliverables

To fit four weeks we drop the CESNET real-drift test and a separate vendor-specific-drift test, and keep the proof minimal.

Risks and fallbacks

Risk	Fallback
Novelty is partial (CIoTA is close); some 2026 papers are preprints	Claim the specific combination and win test 2; verify numbers against the PDFs before citing
Peer concurrency on N-BaIoT is synthetic	Let the real ESP32 router change carry the real-drift claim
Multi-device plumbing or on-board features slip	Ship the simpler variant that learns threat directions from its own alarms; use replay mode for cost; the freeze-attack study stands alone
The multi-device proof is harder than it looks	State it as an assumption plus the matching negative result

REFERENCES

Compiled references, newest first

36 references, 26 from 2025 to 2026. Full list with links, reading status and one line on why each matters: **references.md** *in the repository.* **Read** *says how much we actually read; preprints are not peer-reviewed, so check numbers against the PDF.*

Year	Paper	Type	Read
2026	Feature-Context Consistency	preprint	full
2026	The Price of Self-Calibration	preprint	not read
2026	Navigating the Latent Manifold	preprint	full
2026	CAPTURE: Disentangling Preference Drift from Memory Poisoning in Personaliz...	preprint	not read
2026	Candidate Comparability Before Promotion	preprint	abstract
2026	Empirical Analysis of Evasion and Poisoning Against Malware Data Drift Dete...	preprint	full
2026	Self-Poisoning in Adaptive Out-of-Distribution Detection	preprint	full
2026	Theory of Continual Learning Against Data Poisoning Attacks	preprint	full
2026	Rapid Poison: Practical Poisoning Attacks Against the Rapid Response Framework	peer-reviewed	full
2026	What changes after deployment? A survey on On-device Learning in TinyML	preprint	full
2026	When Normality Shifts: Risk-Aware Test-Time Adaptation for Unsupervised Tab...	preprint	full
2026	The Boiling Frog Threshold	preprint	full
2026	ACORN-IDS: Adaptive Continual Novelty Detection for Intrusion Detection Sys...	preprint	full
2026	Online robust covariance matrix estimation and outlier detection	preprint	full
2026	Contrastive Adversarial Agents for Intentional Drift Induction in Network I...	peer-reviewed	full
2026	Strategic Games and Zero Shot Attacks on Heavy-Hitter Network Flow Monitoring	peer-reviewed	full
2025	Provably Safe Model Updates	preprint	full
2025	Adversarial Habituation Attack	peer-reviewed	abstract
2025	Shift Detection and Adaptation for Network Intrusion Detection	preprint	full
2025	CITADEL: Continual Anomaly Detection for Enhanced Learning in IoT Intrusion...	preprint	not read
2025	CND-IDS: Continual Novelty Detection for Intrusion Detection Systems	peer-reviewed	full
2025	On the Adversarial Risk of Test Time Adaptation	peer-reviewed	full
2025	Provably Reliable Conformal Prediction Sets in the Presence of Data Poisoning	peer-reviewed	partial
2025	Adversarial Attacks for Drift Detection	peer-reviewed	partial
2025	**CORE** CESNET-TimeSeries24: Time Series Dataset for Network Traffic Anomaly Detect...	peer-reviewed	dataset
2025	Network Intrusion Datasets	peer-reviewed	partial
2024	Data Poisoning Attack against Neural Network-Based On-Device Learning Anoma...	peer-reviewed	full
2024	AOC-IDS: Autonomous Online Framework with Contrastive Learning for Intrusio...	peer-reviewed	full
2023	Anomaly Detection in the Open World	peer-reviewed	full
2022	Adversarial concept drift detection under poisoning attacks for robust data...	peer-reviewed	full
2018	CIoTA: Collaborative IoT Anomaly Detection via Blockchain	peer-reviewed	partial
2018	**CORE** N-BaIoT: Network-Based Detection of IoT Botnet Attacks Using Deep Autoencoders	peer-reviewed	dataset
2018	**CORE** Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection	peer-reviewed	full
2017	Machine Learning with Adversaries	peer-reviewed	full
2012	**CORE** Security Analysis of Online Centroid Anomaly Detection	peer-reviewed	full
2009	**CORE** ANTIDOTE: Understanding and Defending against Poisoning of Anomaly Detectors	peer-reviewed	full